Understanding Quantum Parallelism through Programming

Janche Sang
Dept. of Computer Science
Cleveland State University
Cleveland, OH, USA

Chansu Yu
Dept. of Electrical and Computer Engineering
Cleveland State University
Cleveland, OH, USA

2023 International Conference on Computational Science and Computational Intelligence (CSCI) | 979-8-3503-6151-3/23/$31.00 ©2023 IEEE | DOI: 10.1109/CSCI62032.2023.00177

***Abstract*—Quantum parallelism arises from the ability of quantum qubits to exist in multiple states simultaneously due to superposition. It enables quantum computers to explore a vast number of possibilities in parallel, potentially solving complex problems exponentially faster than classical computers. Recently, there has been an increasing demand for skilled professionals with expertise in quantum computing and programming. This paper presents a practical approach to teaching quantum computing to Electrical Engineering & Computer Science (EECS) students through dedicated hands-on programming labs. The labs cover a diverse range of topics, encompassing fundamental elements, such as quantum gates and circuits, as well as quantum algorithms. As educators, we aim to share our teaching insights and resources with fellow instructors in the field. This article elucidates the rationale behind the design of each experiment, enabling a deeper understanding of quantum computing.**

***Index Terms*—Quantum Parallelism, Quantum Computing Education, Quantum Programming, Hands-On Laboratory, Qiskit**

Type of the submission: Full/Regular Research Paper
Research track: CSCI-RTPD

I. Introduction

Quantum Computing is a rapidly growing field with many potential applications, including cryptography, optimization, pharmaceuticals, materials science, etc. As more and more companies and organizations invest in quantum research and development, the demand for skilled professionals with expertise in quantum computing is likely to increase. That is, having an understanding of quantum computing and practical experiences in quantum programming can be a valuable asset for Electrical Engineering & Computer Science (EECS) students looking to pursue careers in industry or academia. Therefore, in Fall 2022, we started to offer an introductory-level Quantum Computing course the first time for our EECS senior students at Cleveland State University(CSU).

Recommended by Prof. Kiper at Miami University [12], we chose Bernhardt's book *"Quantum Computing for Everyone"* [1] as the textbook. This book provides simple explanations of the challenging mathematics of quantum computing and gives elementary examples to illustrate what it means and how it is applied in problem solving. However, like many other quantum computing books, this textbook also lacks the stuff of practical quantum programming, which would be useful for the EECS students because quantum programming expertise will likely be in high demand as quantum computing continues to advance.

Hands-on programming provides a means to bridge the gap between theoretical knowledge and practical implementation in quantum computing. By engaging in hands-on programming, students can gain a deeper understanding of quantum concepts such as superposition, entanglement, quantum gates and quantum algorithms. For example, it might be easier for students to understand how the Deutsch [5] and the Deutsch-Jozsa [6] algorithms work via the quantum phase kickback circuit implementation than the rigorous proof in math. Furthermore, actually writing code and running it on quantum computers or simulators, students can explore the real-world implications of quantum algorithms and discover novel ways to leverage quantum computing power.

Figure 1 shows a general quantum programming model that encompasses both quantum and classical components. The quantum component comprises the quantum circuits constructed using qubits and quantum gates. Quantum gates, such as Hadamard gates and others, are employed to manipulate the quantum states of qubits, with Hadamard gates being particularly valuable for creating superposition states. As depicted in Figure 1, the Hadamard gates can concurrently generate all 2^n input cases for the quantum circuits. These circuits encode the quantum algorithm and can be repeated based on the specified problem, as exemplified by the Grover's Search algorithm [7].

The classical component serves as the controller of the quantum computation. After each quantum computation round, the classical component gathers the measurement results from the qubits. If deemed necessary, the classical component directs the quantum component to perform more rounds of quantum computation. This process can be repeated within a predefined time frame or until specific convergence criteria are met. Subsequently, the classical component aggregates all the measurement results and processes them to obtain the final computation results, as illustrated by algorithms like Simon's algorithm [18] and Shor's algorithm [17].

Several open-source quantum software development kits, such as Amazon's Braket, Google's Cirq, IBM's Qiskit, Microsoft's Q# and QDK, etc. are available on-line. For the dedicated hands-on labs, we focused on teaching students a single quantum programming environment throughout the course. We chose IBM's Qiskit because IBM is the only

2769-5654/23/$31.00 ©2023 IEEE
DOI 10.1109/CSCI62032.2023.00177

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 24,2026 at 10:04:10 UTC from IEEE Xplore. Restrictions apply.

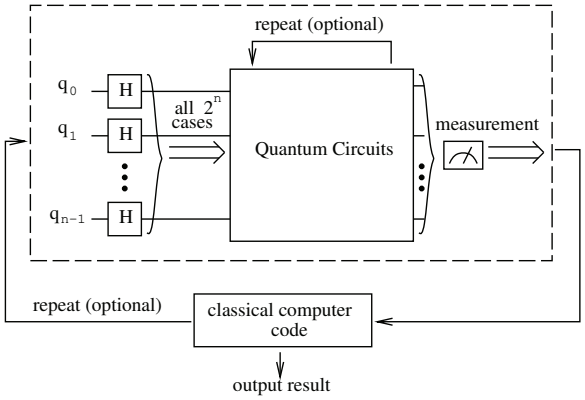


Fig. 1. A General Quantum Programming Model

provider which allows unlimited free accesses to actual quantum computers. Furthermore, Cleveland Clinic [3] installed the IBM Quantum System One [8], which supports a 127-qubit Eagle processor, to accelerate biomedical discoveries. Through the collaboration between CSU and the Cleveland Clinic, our students will have opportunities to conduct experiments on the cutting-edge quantum computer.

Therefore, our goal of this work was to design hands-on experiments for students to implement python-based quantum programs during seven dedicated lab sessions. Many of the experiments are tied to the textbook and students are asked to modify a program template and then verify their experimental result with the corresponding example described in the textbook. Through this report, we would like to share our teaching experience and materials with other instructors teaching quantum computing. Interested readers can find the full handouts and the detailed explanation of each experiment in [10].

II. Related Work

Traditional quantum computing courses usually focus on physical and mathematical concepts and most of them target graduate students. Over the past three years, an increasing number of universities have been offering introductory quantum computing courses for undergraduate students, emphasizing the practical aspects of quantum computing. As described in the paper [4], the instructors gave online lectures worldwide and demonstrated running a set of Qiskit quantum programs with Jupyter notebooks. In [2], the authors compared various quantum programming environments and chose Qiskit as the framework for teaching. They instructed students in various examples, including quantum teleportation, the Deutsch–Jozsa algorithm, the Bernstein–Vazirani algorithm, Simon's algorithm, the Quantum Fourier transform, and others, all of which were supported by Qiskit Quantum Computing Labs web pages. A practical software-driven approach for teaching quantum computing was proposed in [13]. Their students needed to use Microsoft Quantum Katas complete six programming assignments and submit a final programming project. Similarly, the teachers presented their Computer Science-oriented approach in [16] by using approximately 30 Juputer notebooks, focusing on practical concepts and teaching Qiskit-based quantum programming in a three-day-long workshop.

The major difference between our approach and the aforementioned previous work is that we offered dedicated hands-on labs, providing students with in-person access to the instructor or lab TA for assistance when needed. Furthermore, many of the experiments are closely linked to the content of the textbook, and these experiments are designed to be progressive, with each new experiment building upon the concepts and skills learned from the previous ones. This approach allows students to gradually apply their knowledge, ensuring a deeper understanding as they progress through the course.

III. Hands-on Quantum Programming Labs

Our class is scheduled weekly for 3 consecutive hours and a 75-minute hands-on lab section is carried out immediately after the topic has been discussed in the class. Performing a lab immediately after the topic is taught ensures that the information is still fresh in students' minds. During a hands-on lab, students can observe the results of their actions in real-time. This instant feedback allows them to identify and correct any misconceptions or errors, further reinforcing their learning and understanding of the material.

The seven dedicated hands-on labs were conducted in our Linux cluster lab. Each workstation has been installed the software packages and libraries, such as Python3, qiskit, jupyter, etc. Note that the Jupyter Notebook [19] is an interactive computing environment and its cell-based approach makes it easy to organize code and text, run code interactively, and create documents that combine executable code, visualizations, and explanations.

In the lab, students need to follow the procedure stated in the handout step by step to implement the programs, record, and explain the results in their lab reports. This section gives a brief description of each lab and some interesting experiments which are important for students to practice.

A. Hands-on Lab 1: Python and Jupyter

In the first warm-up lab, each student will set up his/her working environment (e.g. installing Qiskit tools, creating a dedicated sub-directory for this course, etc.), and get familiar with some Linux commands. Students will also learn how to implement and run Python 3 programs in Jupyter Notebook's code cells, as well as how to load and execute program templates in cells. Furthermore, they will review some features in the Python language, such as using indentation instead of a pair of curly braces in C/C++/Java to indicate a block of code, the dictionary data structure which uses string as the array index, etc. Note that, in this lab, students are asked to copy and run the quantum "Hello World" program which uses a Hadamard gate to set a qubit into a superposition state and then measures it. However, we will not explain how the

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 24,2026 at 10:04:10 UTC from IEEE Xplore. Restrictions apply.

code works in detail until the next lab. This fosters a sense of curiosity that may lead to more profound and lasting learning outcomes.

B. Hands-on Lab 2: Qiskit and Entanglement

In this lab, students will learn how to write simple quantum programs in Qiskit. A Qiskit program typically follows a structured pattern to firstly construct the quantum circuits using the Qiskit Terra module and then execute the circuits either on a simulator (i.e. the Qiskit Aer module) or on a real IBM quantum computer. Instructors should remind students that the Qiskit's qubits order is simply reversed as compared to the order used in the textbook [1] (also in the reference [20]). That is, Qiskit puts q_0 rightmost (e.g. $q_2q_1q_0$) while the textbook puts q_0 leftmost (e.g. $q_0q_1q_2$). Students have to be careful while reading Qiskit documents, Qiskit gate operation matrix, histogram output result, etc.

Students will study in detail the quantum "Hello World" program template given in the previous lab. Next, they will learn that quantum randomness via the H gate is not simply like a classical random coin toss. That is, as shown in Figure 2, students will modify the template by applying two H gates in succession and observe that the result is not exactly the same as tossing the coin twice. They also need to verify the result through the state vector calculation.

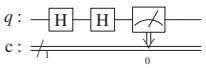


Fig. 2. Applying two H gates in succession

The major theme of Lab 2 is entanglement. Students will construct the two-qubit entanglement circuit shown in Figure 3 and observe the entangled results (i.e. either "00" or "11"). In the next experiment, they need to modify the circuit by initializing the qubit q_1 to be 1 like in the Figure 4. Students are asked to explain whether the result is entangled or not in their reports. They can refer to page 59 in the textbook about the simple rule to check the entanglement of two qubits. That is, assume that r, s, t, and u represent the probability amplitudes of two qubits q_0q_1 states: 00, 01, 10, and 11, respectively. These two qubits are entangled if $ru \neq st$. Otherwise, they are not entangled and can be decomposed into the tensor product of two individual qubits.

C. Hands-on Lab 3: Quantum Gates and Circuits

This lab starts with doing two experiments: one is to entangle three qubits from the same control qubit and the other is to entangle four qubits in a chain manner. Next, students are asked to use quantum gates such as X, H, CNOT, and Toffoli, to construct circuits for emulating the classical AND, NAND, and OR gates. Some students may have learned NP-completeness already. Instructors could tell them that, for NP-complete problems, such as SAT, quantum computers can gives quadratic speedup over the classical computers. Learning how to emulate the classical gates will help them build the oracle circuits used in the Grover's search algorithm [7] for solving the SAT problem. These SAT-related experiments will be conducted in Lab 7 in the future.

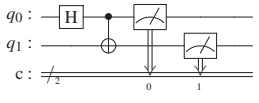


Fig. 3. Two-qubit Entanglement: 00 or 11

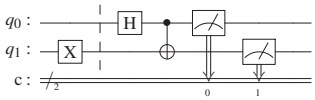


Fig. 4. Two-qubit Entanglement: 01 or 10

To emulate the classical AND gate function, we can simply use a Toffoli gate `ccx(c0,c1,t)`, where `c0` and `c1` are the control qubits and `t` is the target qubit. When both `c0` and `c1` are set to 1, the target qubit `t` will be flipped to 1. To implement the NAND gate function, we just need to add a NOT gate (i.e. the quantum X gate) to flip the target qubit `t` of the Toffoli gate at the end.

Note that there are two different methods to implement the classical OR gate. One is to use two CNOT gates and one Toffoli gate, as shown in Figure 5. If either one of the two inputs is 1, it uses the CNOT to toggle the output result to be 1. If both inputs are 1's, output result will be toggled twice back to 0 and hence we need the Toffoli gate to toggle the output qubit again. The other method is to adopt X and AND gates based on the following equation:

$$q_0 \vee q_1 = \neg(\neg q_0 \wedge \neg q_1)$$

That is, as shown in Figure 6, we invert both inputs and apply the NAND function. Finally, we have to invert the inputs back to their original values.

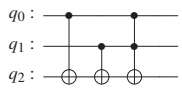


Fig. 5. Implementing the OR gate with CNOT and Toffoli gates

There are two slightly more complicated quantum circuits that are useful for students to study. The first circuit is shown in Figure 7. After implementing and executing the program, students will find out, for the four possible inputs q_0q_1: 00, 01, 10, and 11, the corresponding outputs will be: 00, 10, 01, and

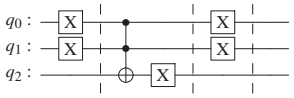


Fig. 6. An alternative implementation of the OR gate

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 24,2026 at 10:04:10 UTC from IEEE Xplore. Restrictions apply.

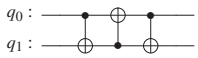


Fig. 7. Swap the states in q_0 and q_1

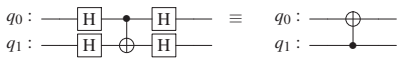


Fig. 8. Phase Kickback

11, respectively. They need to deduce that the circuit swaps the states between q_0 and q_1.

The second circuit, known as Phase Kickback and depicted on the left of Figure 8, will generate the outputs q_0q_1 as 00, 11, 10, and 01, corresponding to the inputs: 00, 01, 10, and 11, respectively. Namely, the circuit acts like the q_1 is the control while the q_0 becomes the target of a CNOT gate, as shown in the right of Figure 8. Note that the two H gates in q_0's wire are not successive due to the presence of a CNOT connection in between. The Phase Kickback circuit has been used in many quantum algorithms. In our experience, comprehending the Phase Kickback circuit would be helpful for beginners in elucidating the workings of the Deutsch and Deutsch-Jozsa algorithms.

It is also worth mentioning that, though not included in the Lab3, the circuits in Figure 7 and Figure 8 can be used together. The current IBM quantum computer adopts the heavy-hexagon lattice topology to connect qubits [14]. Hence, the swap gate is needed to facilitate interactions between non-adjacent (i.e. not directly connected) qubits. Moreover, the control qubits are not adjacent in the heavy-hexagon lattice and this limitation makes the circuit in Figure 7 impossible if q_0 and q_1 are directly connected together. If the qubit q_1 in Figure 7 cannot be the control qubit, we can simply use the Phase Kickback to solve the problem [21]. The solution is depicted in the rightmost part in Figure 9. This can be left as a homework exercise for students.

D. Hands-on Lab 4: Quantum Key Distribution

Quantum Key Distribution (QKD) is a cryptographic technique that utilizes quantum mechanics to enable two communication parties to produce a shared random secret key known only to them. The secret key can be used for encrypting and decrypting messages. There are two QKD protocols discussed in the textbook: BB84 and Ekert91. In this lab, students are asked to implement Qiskit programs to emulate the behaviors of these two protocols and verify the output results of

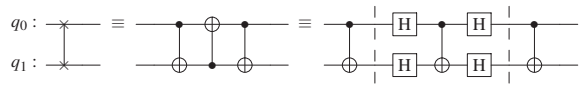


Fig. 9. Swap the states in q_0 and q_1 if q_1 cannot be the control qubit

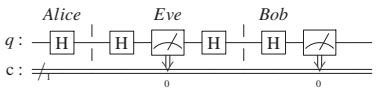


Fig. 10. BB84 emulation with Eve's eavesdropping and guessing right

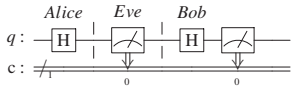


Fig. 11. BB84 emulation with Eve's eavesdropping and guessing wrong

the probabilities from their programs with the corresponding descriptions in the textbook.

The first simple experiment is to let students observe that Alice and Bob will obtain the same qubit value if they choose the same basis – either the vertical or horizontal direction. In the next experiment, students are asked to run a template program which emulates the BB84 protocol except detecting any interception. The template initially sets the n to be 4, a small value so students can trace the executions easily. Based on the program output, they need to mark, in their report, the bit positions where Alice and Bob use the same bases and then extract the corresponding bits from "Alice bits" and "Bob results" into two bit sequences. Finally, they need to add the code to select half of the same-basis bits and compare them whether the same or not. If the bits comparison is True, students can increase the variable n to a larger value, say 1000, and run the program again to make sure that their program works for a larger size of bit sequence.

In the third experiment, students are asked to add lines of code to their program implemented in the first experiment to emulate Eve's interception. It can be observed that if Alice, Eve, and Bob choose the same basis, they will all get the same bit and hence Eve's eavesdropping cannot be detected. Figure 10 is an example in which Alice, Eve, and Bob all choose the horizontal basis. Note that after Eve guesses the measurement basis, she will send the qubit to Bob using the same basis. If Eve chooses the wrong basis, as shown in Figure 11, the emulated program will output that Bob gets the right bit with only a 50% chance.

The last three experiments are about the Ekert91 protocol which utilizes entangled qubits as in the Bell's test. There are a few variations of the protocol. Based on the version presented in the textbook, there are $3n$ pairs of entangled qubits. For each pair, Alice receives one and randomly chooses a direction to measure her qubit, while Bob gets the other and also randomly selects a direction to measure it. There are three directions Alice and Bob can use: 0°, 120°, or 240°. Therefore, they will agree around n bits when both choose the same direction. Because the bases they chose will be revealed over a public insecure line, they can use these same-basis n bits as the shared secure key if Eve is not intercepting. Note that to detect interception, they use the sequence of qubits that come from the times when they chose different bases. Detailed

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 24,2026 at 10:04:10 UTC from IEEE Xplore. Restrictions apply.

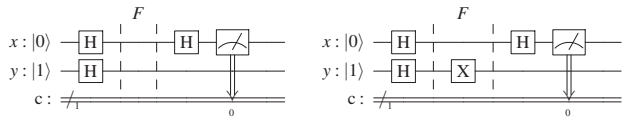


Fig. 12. Deutsch -– the Constant 0 function (left) and the Constant 1 function (right)

explanations of these three experiments can be found in [10].

E. Hands-on Lab 5: Deutsch and Deutsch-Jozsa Algorithms

The Deutsch algorithm is the first algorithm to show that a quantum algorithm could be faster than a classic one to solve a specific problem. It specifically addresses the task of determining whether a black-box function $f : \{0,1\} \longrightarrow \{0,1\}$ is constant or balanced. Note that a function is called balanced if it sends half of its inputs to 0 and the other half to 1. There are four such functions. Two are constant: the constant 0 function or the constant 1 function , while the other two are balanced: the identity function, or the inversion function. In classical computing, it would typically require two function evaluations to tell whether a black-box function is constant or balanced. By employing quantum principles such as superposition and interference, the quantum algorithm requires only a single query to the function, providing a speedup over the classical method.

In the first four experiments of this lab, students will learned how to construct the circuits for each of the four black-box functions. For a quantum system, every operation must be unitary and thus reversible. For achieving this, each function, say F, needs another input y to map the state $|x,y\rangle$ to the state $|x,y \oplus f(x)\rangle$. Because $|x,(y \oplus f(x)) \oplus f(x)\rangle = |x,y \oplus 0\rangle = |x,y\rangle$, the function F is its own inverse. To construct the circuit for the constant 0 function, where $f(x) = 0$, the output of the bottom qubit becomes $y \oplus f(x) = y \oplus 0 = y$. Namely, there are only two straight wires in the black box F and there is no connection between the qubit x and the qubit y, as shown in the left part of Figure 12. Hence, the top qubit x goes through two H gates in succession. Based on the experience we learned from the Experiment 2 in the Lab 2, the qubit x will be restored back to its initial value 0 before measurement. Similarly, the circuit for the constant 1 function, where $f(x) = 1$, can be constructed as in the right part of Figure 12. This is because $y \oplus f(x) = y \oplus 1 = \bar{y}$. Same as the constant 0 function, there is no connection between the qubit x and the qubit y. The qubit x also goes through two successive H gates and its value will be restored back to its initial value 0. Therefore, both constant functions yield the value 0 in the qubit x.

The left portion of Figure 13 shows how to build the circuit of the balanced identity function, where $f(x) = x$. That is, the function F adopts a CNOT gate to map the input state $|x,y\rangle$ to the output state $|x,y \oplus f(x)\rangle = |x,y \oplus x\rangle$. From the Experiment 7 in the Lab 3 (i.e. the phase kickback circuit), we know that bottom qubit y becomes the control bit of the CNOT gate. Furthermore, another tricky setting is the initial value of the qubit y. The value 1 in y will let the qubit y toggle the value

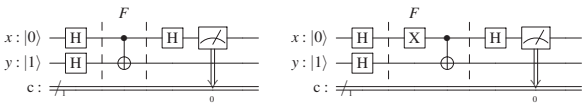


Fig. 13. Deutsch — the balanced functions; either Identity (left) or Inversion (right)

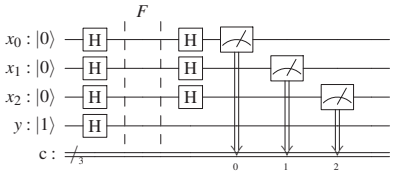


Fig. 14. Deutsch-Jozsa – the constant 0 function $f(x_0,x_1,x_2) = 0$

in the qubit x from 0 to 1, which is different than the result of the constant functions. Likewise, the circuit of the balanced inversion function, where $f(x) = \bar{x}$, can be constructed as in the right portion of Figure 13. The function F maps the state from $|x,y\rangle$ to $|x,y \oplus f(x)\rangle = |x,y \oplus \bar{x}\rangle$. To implement it, we need to apply an X gate for the qubit x to invert its value. However, this X gate will not change the superposition state $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ in x because $|0\rangle$ becomes $|1\rangle$ and $|1\rangle$ becomes $|0\rangle$. Same as the balanced identity function, the value 1 in y will flip the value in the qubit x from 0 to 1. Hence, if we measure the qubit x and its value is 1, we know that the function is balanced.

In the class lecture, students learned that the Deutsch algorithm doesn't truly demonstrate the power of quantum parallelism because it employs a single query to distinguish between two cases: constant or balanced. The Deutsch-Jozsa algorithm generalizes the Deutsch problem with a larger domain. It determines whether a black-box function $f : \{0,1\}^n \longrightarrow \{0,1\}$ is constant (i.e. it always outputs the same value for all 2^n inputs) or balanced (i.e. half of the inputs go to 0 and the other half go to 1). The Deutsch-Jozsa algorithm shows the power of quantum parallelism by using only one query to the oracle, as compared to the worst case scenario $2^{n-1}+1$ function evaluations needed in the classical method.

Students are asked to construct the circuits for the constant 0 function and for the balanced function $f(x_0,x_1,x_2) = x_0 \oplus x_1 \oplus x_2$, as shown in Figure 14 and Figure 15, respectively. For the constant 0 function, each of the three qubits $x_0x_1x_2$ goes through two successive H gates and the value in $x_0x_1x_2$ will be restored back to "000". For the balanced function, there is at least one CNOT connection between the x qubits and the qubit y. Since the phase kickback causes the input x qubits changed through the connection(s), the value in $x_0x_1x_2$ will not be "000" anymore.

We believe that these hands-on programming experiments will help students easily understand the Deutsch and the Deutsch-Jozsa algorithms from a different perspective, rather than relying solely on the rigorous mathematical proofs presented in the textbook. The phase kickback circuit, along with the y qubit's initial value set to 1, will flip certain values of the x-qubit for the balanced functions. This flipping will not occur

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 24,2026 at 10:04:10 UTC from IEEE Xplore. Restrictions apply.

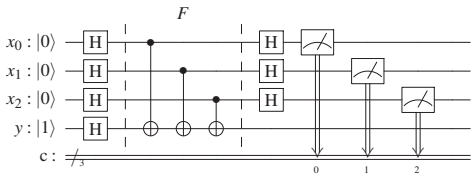


Fig. 15. Deutsch-Jozsa – the balanced function $f(x_0,x_1,x_2) = x_0 \oplus x_1 \oplus x_2$

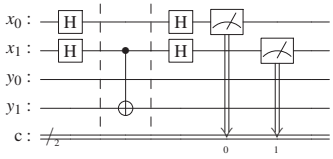


Fig. 16. A Simple Example of Simon's Algorithm Circuit with $s = 10$

for the constant functions due to the lack of any connection.

F. Hands-on Lab 6: Simon's Algorithm

The Simon's algorithm solves the problem of finding the hidden string in a black-box function $f : \{0,1\}^n \longrightarrow \{0,1\}^n$. The black-box 2-to-1 function has the property that there is a secret non-zero binary string s of length n, (i.e.$s \neq 00\ldots0$), s.t. $f(x) = f(y)$ if and only if $y = x$ or $y = x \oplus s$. The Simon's algorithm contains quantum procedures as well as classical post-processing procedures. As described in the textbook, we can run Simon's circuit $n+N$ times, where N does not depend on n, to get $n+N$ equations. These equations contains the $n-1$ independent vectors, so we can use a classical algorithm (e.g. Gaussian elimination) to recover the secret string s.

Students begin this lab by running a 2-qubit template program with the hidden string $s = 10$. Figure 16 depicts the circuit of this program. This template outputs the top x qubits either 00 or 01. Note that the template does not call Gaussian elimination to find s and hence students need to manually solve the $n-1$ linearly independent equations. This will help them know how the whole algorithm works. Because the output string '00' which wouldn't give us any information, students need to use the other output string 01 to calculate its dot product with the secret string s (i.e. $s_0 s_1$) and the result should be 0. That is, they will get

$$0 \times s_0 + 1 \times s_1 = 0.$$

They have to figure out that $s_1 = 0$ and hence $s = 10$ because not all of the digits in s can be 0.

In this lab, students will also learn a simple method, as presented in [15], for constructing an oracle circuit F using a given string s. Figure 17 shows the steps to construct such a 2-to-1 mapping function. Note that in the first step, because each qubit in the register y is initialized to be 0, the content of each qubit in the register x, which is the classical information encoded as either a $|0\rangle$ or a $|1\rangle$, will be copied through the CNOT gate to the corresponding qubit in the register y . In the second step, if $x_k == 0$, the register y remains unchanged. Otherwise, the register y will be XORed with s, i.e. $y \longleftarrow y \oplus s$. Figure 18 illustrates an example of using these two steps to

1) Apply the CNOT gates from qubits of the first register (i.e. x) to qubits of the second register (i.e. y).
2) Because the bits in the string s cannot be all 0, find the least index k such that $s_k = 1$. Next, apply the CNOT gates from the qubit x_k to any qubit y_i if $s_i = 1$.

Fig. 17. A Simon's Oracle Construction Algorithm

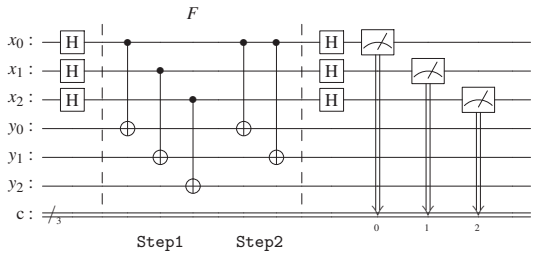


Fig. 18. An Example of Constructing Simon's Algorithm Oracle Circuit with $s = 110$

construct the Simon's Algorithm Oracle Circuit with $s = 110$. Note that in this example, k is 0 because $s_0 = 1$. If the secret string s is 011, k is 1 since $s_1 = 1$ and hence the CNOT control bit is x_1 in the Step 2.

G. Hands-on Lab 7: Grover's Search Algorithm

As described in the textbook, Grover's search algorithm is a quantum computing technique designed to accelerate the search for the specific item(s) in an unstructured database. In the algorithm, an initial quantum state is prepared by applying a Hadamard transform to all possible input states. Subsequently, a specialized quantum oracle is employed to mark the target state by inverting its phase (i.e. flipping the sign of its probability amplitude). The next step involves a process of applying a diffusion operator, which amplifies the probability amplitude of the target state while reducing the amplitudes of other states. The phase inversion oracle and the amplitude amplification (also called amplitude magnification) procedure may need to be applied repeatedly approximately $\sqrt{m}$ times, where m is the number of items in the database. For example, in the case of an NP-Complete problem 3-SAT with n bits, there are $m = 2^n$ possible assignments in the database. The Grover's search algorithm can reduce the run-time complexity from $O(2^n)$ to $O(\sqrt{m})$, which is equivalent to $O(2^{n/2})$. While quadratic speedup may not be as impressive as exponential speedup, it remains valuable for handling massive data sets.

The first experiment in this lab asks students to load and run a template program which finds the desired element location 11 among 2-bit strings: 00, 01, 10, and 11. In fact, this is a very simple SAT problem that determines the satisfiability of the expression $q_0 \wedge q_1$. Figure 19 shows the circuit which includes the initial Hadamard transforms, the phase inversion oracle, and the amplitude amplification. The phase inversion oracle can be realized using a Controlled-Z gate `cz(c,t)`, which

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 24,2026 at 10:04:10 UTC from IEEE Xplore. Restrictions apply.

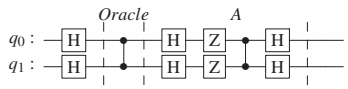


Fig. 19. Grover's Algorithm with the Boolean expression $q_0 \wedge q_1$

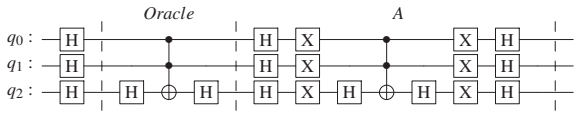


Fig. 20. Grover's Algorithm with the Boolean expression $q_0 \wedge q_1 \wedge q_2$

flips the phase of the target qubit `t` if the control qubit `c` is in the $|1\rangle$ state. As described in [11], the Controlled-Z gate is a phase-logic operation which performs phase AND (denoted as pAND). In this experiment, students are asked to use the Operator class defined in the Qiskit `quantum_info` library in their program to display the oracle operation matrix and the amplitude amplification matrix. They will find out that the operation matrix of amplitude amplification and check whether it is the same as in the textbook.

In the next experiment, students will load and run a template program that includes a 3-bit oracle with the desired element location at 111. Because Qiskit does not support the ccz gate, the template program uses a ccx gate instead, but it requires placing a pair of H-gates before and after the X gate in the ccx operation. It's important to note that operation of HXH is equivalent to a Z gate. The program outputs two distinct probability amplitudes. The higher one is located at 111, while the lower one is situated as other locations. Students are asked to compare these two probability amplitudes with the two probability amplitudes mentioned in the textbook. As mentioned earlier, in order to achieve a higher probability of obtaining the correct answer, it may be necessary to apply the oracle and amplitude amplification repeatedly. To repeat the process, we do not recommend that students compose the oracle and amplitude amplification circuits again either manually or via a loop construct in Python. This is because if the transpiler does not aware of the repetition, it may lead the number of gates growing exponentially to $O(2^{n/2})$. Therefore, students are instructed to adapt the `repeat()` method in Qiskit, which prevents the transpiler from adding the same circuits repeatedly. It will be interesting for students to observe that if the oracle and the amplitude amplification tasks are performed three times, the probability amplitude of the desired location will decrease. In other words, it is overcooked.

IV. Conclusion and Future Work

A collection of engaging laboratory experiments has been developed to serve as supplementary resources for the textbook. Many of the experiments are closely linked with the content of the textbook. These experiments capture students' interest while also offering a concrete platform for validating their comprehension through the comparison of experimental outcomes with textbook examples. The feedback collected from the students has, overall, been very positive. The students indicated that these practical labs help them not only improve their hands-on skills in quantum programming, but also understand the rationale behind them. We believe that this report is valuable for utilization as an instructor's manual for Quantum programming education.

In the future, we plan to incorporate additional quantum programming experiments, such as the bridge gate [9], satisfiability problem with a slightly more complex Boolean formula, Shor's algorithm [17], and others, into the hands-on labs.

References

[1] C. Bernhardt. *Quantum Computing for Everyone*. The MIT Press, Cambridge, Massachusetts, USA, 2020.

[2] G. Carrascal, A. del Barrio, and G. Botella. First experiences of teaching quantum computing. *The Journal of Supercomputing*, Vol. 77:2770–2799, 2021.

[3] Cleveland Clinic. Quantum Computing. https://my.clevelandclinic.org/research/computational-life-sciences/discovery-accelerator/quantum-computing (last accessed 2023-06-10).

[4] E.F. Combarro, S. Vallecorsa, L. J. Rodríguez-Muñiz, A. Aguilar-González, J. Ranilla, and A. Di Meglio. A report on teaching a series of online lectures on quantum computing from CERN. *The Journal of Supercomputing*, Vol. 77:14405–14435, 2021.

[5] D. Deutsch. Quantum theory, the Church-Turing Principle and the universal quantum computer. In *Proceedings of the Royal Society of London, Series A*, 1985.

[6] D. Deutsch and R. Jozsa. Rapid solutions of problems by quantum computation. In *Proceedings of the Royal Society of London, Series A*, 1992.

[7] L. K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on the Theory of Computing*, 1996.

[8] IBM. Quantum System One. https://www.ibm.com/quantum/systems (last accessed 2023-04-10).

[9] T. Itoko, R. Raymond, T. Imamichi, and A. Matsuo. Optimization of Quantum Circuit Mapping using Gate Transformation and Commutation. *Integration, the VLSI journal*, Vol. 70:43–50, 2020.

[10] J. Sang and C. Yu. Hands-on Quantum Programming Labs for EECS Students. https://arxiv.org/pdf/2308.14002.pdf (last accessed 2023-09-29).

[11] E. Johnston, N. Harrigan, and M. Gimeno-Segovia. *Programming Quantum Computers*. O'Reilly Media, Inc., Sebastopol, CA, USA, 2019.

[12] J. Kiper. CSE 470N Course Syllabus. Miami University, Spring 2022.

[13] M. Mykhailova and K. M. Svore. Teaching Quantum Computing through a Practical Software-driven Approach: Experience Report. In *SIGCSE '20: The 51st ACM Technical Symposium on Computer Science Education*, 2020.

[14] P. Nation, H. Paik, A. Cross, and Zaira Nazario. The IBM Quantum heavy hex lattice. https://research.ibm.com/blog/heavy-hex-lattice (last accessed 2022-12-10).

[15] R. Raymond. The Simon Algorithm. https://notebook.community/antoniomezzacapo/qiskit-tutorial/community/algorithms/simon_algorithm (last accessed 2022-07-22).

[16] Ö. Salehi, Z. Seskir, and İ. Tepe. A Computer Science-Oriented Approach to Introduce Quantum Computing to a New Audience. *IEEE Transactions on Education*, Vol. 65:1–8, 2022.

[17] P.W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science, IEEE Computer Society*, 1994.

[18] D.R. Simon. On the Power of Quantum Computation. *SIAM Journal on Computing*, Vol. 26, 1997.

[19] The Jupyter Notebook. User Documentation. https://jupyter-notebook.readthedocs.io/en/stable/notebook.html (last accessed 2023-06-10).

[20] N. S. Yanofsky and M. A. Mannucci. *Quantum Computing for Computer Scientists*. Cambridge University Press, Cambridge, England, 2008.

[21] A. Zulehner, A. Paler, and R. Wille. An Efficient Methodology for Mapping Quantum Circuits to the IBM QX Architectures. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pages 1226 – 1236, 2019.

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 24,2026 at 10:04:10 UTC from IEEE Xplore. Restrictions apply.